

Implementation and Performance Evaluation of an NFT Marketplace on the Solana Blockchain

Chapter 1: Introduction

1.1 Background

The increasing adoption of blockchain technology has transformed digital ownership through the emergence of Non-Fungible Tokens (NFTs). NFTs enable the representation of unique digital assets with verifiable ownership stored permanently on a decentralized ledger. Their applications span digital art, gaming, collectibles, intellectual property, and virtual environments, creating a growing demand for efficient decentralized marketplaces capable of supporting secure asset trading.

Traditional blockchain platforms often experience limitations related to transaction throughput, confirmation latency, and increasing transaction costs during periods of high network activity. These challenges directly affect user experience in decentralized marketplaces, where rapid transaction execution and low operational costs are essential.

Solana is a high-performance Layer-1 blockchain specifically designed to address these scalability challenges. Unlike conventional blockchain architectures that process transactions sequentially, Solana combines **Proof of History (PoH)** with **Proof of Stake (PoS)** to provide highly efficient transaction ordering and parallel execution. These architectural innovations enable Solana to achieve high transaction throughput while maintaining low transaction fees.

In this project, an NFT marketplace was implemented on the Solana blockchain using the **Anchor Framework** and the **Rust programming language**. The marketplace supports NFT minting, listing, purchasing, and delisting while demonstrating how Solana's account-based execution model can efficiently manage decentralized digital asset trading.

1.2 Motivation

The primary motivation for selecting Solana lies in its exceptional transaction throughput and low operational cost.

Unlike many blockchain platforms that depend heavily on sequential transaction processing, Solana introduces Proof of History as a cryptographic clock that establishes transaction ordering before consensus. This significantly reduces validator communication overhead and improves network efficiency.

The Anchor Framework further simplifies Solana smart contract development by providing structured account validation, automatic serialization, and reduced boilerplate code. These features improve both developer productivity and application reliability.

Together, Solana's high-performance architecture and developer tooling make it an attractive platform for implementing decentralized NFT marketplaces capable of supporting large transaction volumes.

1.3 Objectives

The objectives of this report are:

- Develop an NFT marketplace using Rust and the Anchor Framework.
- Implement NFT minting, listing, purchasing, and delisting functionality.
- Deploy the marketplace on the Solana Devnet.
- Validate successful execution of marketplace operations.
- Evaluate transaction latency, throughput, and on-chain execution cost.
- Assess Solana's suitability for decentralized NFT marketplace applications.

1.4 Scope of the Report

This report focuses exclusively on the implementation and evaluation of the NFT marketplace developed on the Solana blockchain. It presents the marketplace architecture, deployment process, functional validation, and benchmark results obtained from the Solana Devnet.

Security auditing and cross-platform comparisons are discussed separately within the project's Security Audit Report and Comparative Performance Analysis Report.

Chapter 2: Overview of the Solana Blockchain

2.1 Introduction to Solana

Solana is a high-performance Layer-1 blockchain designed to provide low-latency transaction processing, high throughput, and minimal transaction fees. Unlike conventional blockchain platforms that rely solely on consensus for transaction ordering, Solana introduces **Proof of History (PoH)** as a cryptographic timestamping mechanism that enables validators to agree on transaction order before consensus execution.

The blockchain combines Proof of History with Proof of Stake to improve scalability while preserving decentralization and network security. This architecture enables Solana to process large numbers of transactions efficiently, making it well suited for decentralized finance, gaming applications, and NFT marketplaces.

Smart contracts on Solana are implemented as executable programs written primarily in Rust and deployed to the blockchain. User data is stored separately in accounts, allowing programs to remain stateless while interacting with multiple accounts during transaction execution.

2.2 Account-Based Architecture

Unlike object-oriented storage in Sui or resource-based storage in Aptos, Solana utilizes an **account-based execution model**.

Every account on the blockchain stores either executable program code or user data. Programs modify account data only when the required accounts are explicitly provided within a transaction.

This architecture provides several advantages:

- Efficient account access.
- Reduced global state contention.
- Parallel execution of independent transactions.
- Fine-grained permission control.

The NFT marketplace developed in this project stores marketplace information within dedicated accounts managed by the Anchor framework.

2.3 Anchor Framework

The marketplace was developed using the **Anchor Framework**, one of the most widely adopted development frameworks for Solana smart contracts.

Anchor provides:

- Automatic account validation.
- Program Derived Address (PDA) management.
- Instruction serialization.
- Simplified error handling.
- Reduced boilerplate code.

These features significantly simplify the development of secure Solana applications while improving code maintainability.

2.4 Proof of History and Parallel Processing

Proof of History acts as a decentralized cryptographic clock that records the chronological ordering of events before consensus occurs.

Because validators already agree on transaction ordering, consensus becomes significantly more efficient.

Additionally, Solana's runtime executes transactions in parallel whenever they operate on different accounts.

This architecture enables substantially higher scalability than purely sequential execution models.

2.5 Smart Contract Modules

The NFT marketplace consists of two primary programs.

NFT Program

Responsible for NFT creation and metadata initialization.

Marketplace Program

Responsible for:

- Listing NFTs.

- Purchasing NFTs.
- Delisting NFTs.
- Managing marketplace state.

Together these programs implement the complete NFT trading lifecycle.

2.6 Why Solana for NFT Marketplaces?

Solana provides several advantages for NFT marketplace development.

- Extremely low transaction fees.
- Fast transaction confirmation.
- High throughput architecture.
- Parallel execution.
- Mature Rust development ecosystem.
- Strong Anchor framework support.

These features enable efficient decentralized marketplace applications capable of handling large user populations.

Chapter 3: NFT Marketplace Design and Implementation

3.1 System Overview

The NFT marketplace developed on the Solana blockchain provides a decentralized platform that enables users to mint, list, purchase, and delist Non-Fungible Tokens (NFTs). The implementation utilizes the **Rust programming language** together with the **Anchor Framework**, allowing secure smart contract development while taking advantage of Solana's high-performance execution environment.

The marketplace is designed around Solana's account-based architecture, where NFTs, marketplace listings, and user accounts are represented through on-chain accounts managed by smart contracts. Each marketplace operation is executed through a dedicated program instruction that validates user permissions, updates account state, and records the transaction on the blockchain.

The marketplace supports four primary operations:

- NFT Minting
- NFT Listing
- NFT Purchase
- NFT Delisting

These operations collectively implement the complete lifecycle of NFT ownership and marketplace trading.

Figure 3.1 illustrates the overall architecture of the developed NFT marketplace.

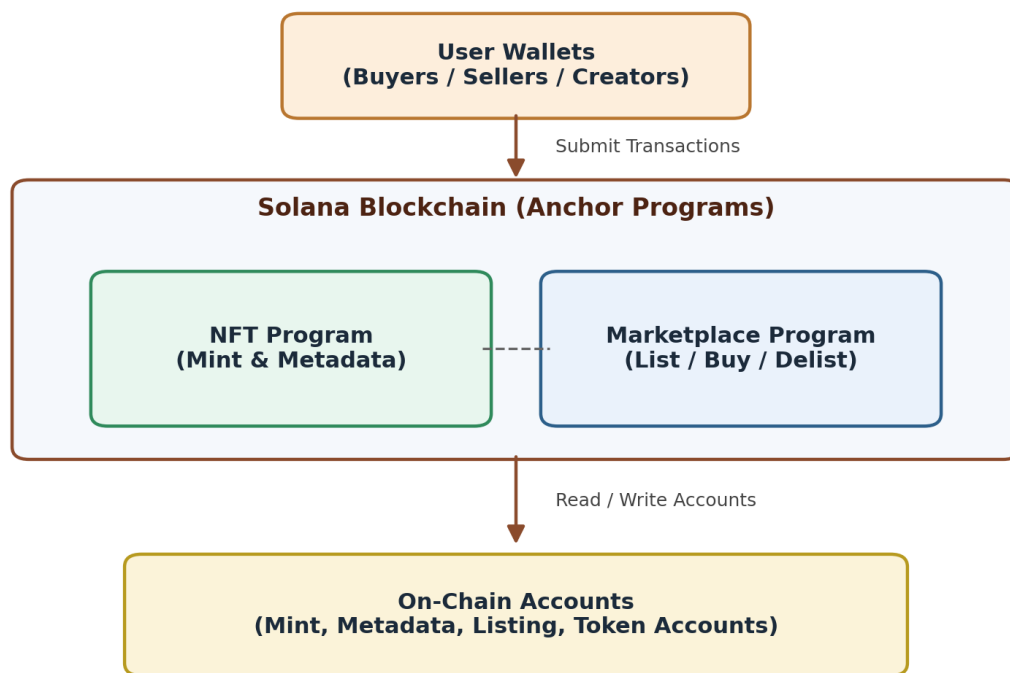


Figure 3.1 Overall Architecture of the NFT Marketplace on Solana

The architecture consists of user wallets interacting with Solana programs deployed on the blockchain. Marketplace instructions modify NFT metadata accounts, listing accounts, and ownership records while preserving transaction consistency throughout every marketplace operation.

3.2 Smart Contract Architecture

The marketplace implementation is organized into multiple program instructions that collectively manage NFT creation and marketplace functionality. The modular design improves readability, simplifies maintenance, and allows future extensions without significantly modifying existing program logic.

The major functional components include:

- NFT Mint Instruction
- Marketplace Listing Instruction
- NFT Purchase Instruction
- NFT Delisting Instruction

Each instruction performs a well-defined task while interacting with specific Solana accounts required for transaction execution.

Figure 3.2 illustrates the interaction among these marketplace components.

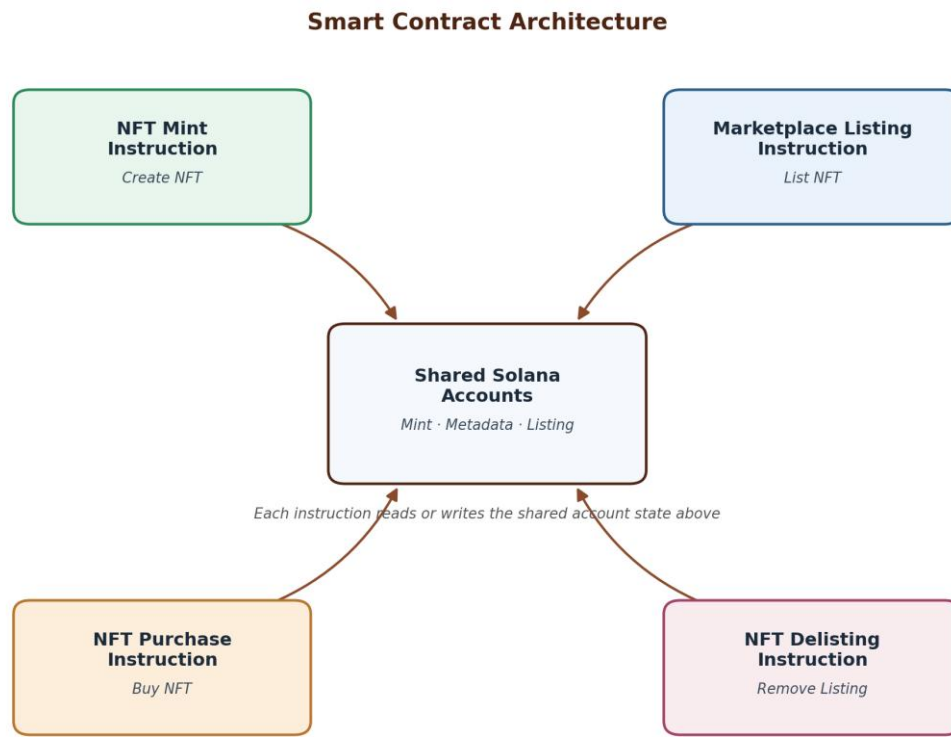


Figure 3.2 Smart Contract Architecture

Unlike monolithic contract designs, separating marketplace functionality into individual instructions improves modularity and facilitates easier debugging and testing.

3.3 NFT Minting

The NFT minting instruction is responsible for creating new NFTs and assigning initial ownership to the creator.

When a user submits a mint transaction, the program performs several validation steps before creating the NFT:

1. Validate the creator's wallet.
2. Generate the NFT mint account.
3. Initialize metadata.
4. Assign ownership.

5. Store NFT information on-chain.

Each NFT contains metadata describing the digital asset, including its unique identifier, creator information, and ownership details.

Because Solana separates executable programs from account data, the NFT itself is represented through dedicated blockchain accounts rather than being embedded directly within the program.

After successful execution, ownership of the NFT is assigned to the creator's wallet, enabling future marketplace interactions.

3.4 Marketplace Listing

The marketplace listing instruction allows NFT owners to offer their assets for sale.

Before creating a listing, the marketplace verifies that the requesting wallet is the legitimate owner of the NFT. Following successful verification, listing information is stored in a dedicated marketplace account containing:

- NFT identifier
- Seller address
- Listing price
- Listing status

The listing account enables potential buyers to discover available NFTs while maintaining separation between NFT ownership data and marketplace information.

The listing workflow consists of the following steps:

1. Seller selects an NFT.
2. Ownership verification is performed.
3. Listing account is created.
4. Sale price is recorded.
5. Marketplace becomes publicly accessible.

If ownership validation fails, the transaction is rejected without modifying marketplace state.

3.5 NFT Purchase

The purchase instruction transfers ownership of an NFT from the seller to the buyer while updating marketplace information.

The transaction executes atomically and performs the following operations:

1. Verify the marketplace listing.
2. Confirm buyer payment.
3. Transfer NFT ownership.
4. Transfer payment to the seller.
5. Remove the completed listing.

Atomic execution guarantees that ownership transfer and payment either both succeed or both fail, preventing inconsistent marketplace states.

Because Solana validates all required accounts before execution, unauthorized transfers or incomplete ownership updates cannot occur.

3.6 NFT Delisting

The marketplace also supports removing active listings.

The delisting instruction allows sellers to withdraw NFTs from the marketplace without transferring ownership.

The process consists of:

- Seller authentication.
- Ownership verification.
- Removal of listing account.
- Marketplace update.

After successful execution, the NFT remains fully owned by the seller while no longer appearing in marketplace listings.

This operation enables sellers to manage marketplace inventory without affecting NFT ownership.

3.7 Account Management

A distinguishing feature of Solana is its account-based execution model.

Every marketplace operation explicitly specifies the accounts involved in the transaction. These include:

- User wallet accounts
- NFT metadata accounts
- Mint accounts
- Marketplace listing accounts
- Token accounts
- System program accounts

The Anchor Framework automatically validates these accounts before executing program logic, significantly reducing the possibility of invalid transactions or unauthorized state modifications.

This explicit account model contributes to Solana's ability to execute independent transactions in parallel while maintaining deterministic execution.

3.8 Marketplace Workflow

The complete marketplace workflow illustrates how users interact with the Solana blockchain throughout the NFT trading lifecycle.

The workflow proceeds as follows:

1. User creates a Solana wallet.
2. User mints an NFT.
3. NFT metadata account is initialized.
4. User lists the NFT for sale.
5. Marketplace listing account is created.
6. Buyer purchases the NFT.
7. Payment is transferred.
8. NFT ownership changes.
9. Marketplace listing is removed.

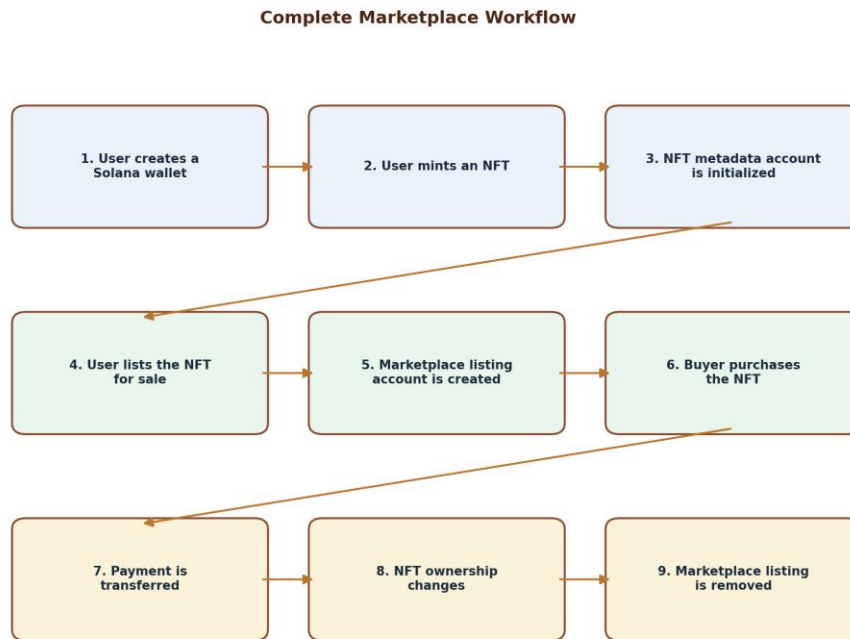


Figure 3.3 Complete Marketplace Workflow

This workflow demonstrates the interaction between user wallets, marketplace accounts, and smart contract instructions while maintaining secure ownership transfer throughout every transaction.

3.9 Design Advantages

The marketplace implementation benefits from several architectural characteristics unique to the Solana blockchain.

First, the account-based architecture allows efficient management of application state by separating executable program logic from user data. This enables programs to remain lightweight while storing marketplace information within dedicated blockchain accounts.

Second, Solana's parallel transaction execution allows independent marketplace operations involving different accounts to execute simultaneously. This significantly improves scalability during periods of high marketplace activity.

The Anchor Framework further enhances reliability by automatically validating accounts, reducing programming complexity, and minimizing common implementation errors associated with manual account handling.

Finally, Solana's low transaction fees make the marketplace economically attractive for users performing frequent NFT trading operations.

Chapter 4: Testnet Deployment and Functional Validation

4.1 Introduction

Following the successful implementation of the NFT marketplace smart contracts, the next phase of the project involved deployment on the **Solana Devnet** and validation of all marketplace functionalities under real blockchain conditions. Deploying the application on a public blockchain provides practical verification that the developed smart contracts operate correctly outside the local development environment while interacting with validators, wallets, and on-chain accounts.

The primary objective of this phase was to confirm that every marketplace operation executes successfully while maintaining data consistency, correct ownership transfer, and reliable transaction execution. Functional validation also ensured that the marketplace behaves as expected before conducting performance benchmarking.

The deployment and testing process included smart contract compilation, program deployment, account initialization, execution of marketplace transactions, and verification of transaction results through the Solana command-line interface.

4.2 Development Environment

The NFT marketplace was developed and deployed using the Solana development ecosystem. The software tools used during implementation and testing are summarized below.

Table 4.1 Development Environment

Component	Description
Programming Language	Rust
Framework	Anchor Framework
Blockchain Network	Solana Devnet
Wallet	Solana CLI Wallet
Development Tool	Anchor CLI
Compiler	Rust Compiler
Benchmark Tool	Custom Benchmark Script
Operating System	macOS

The Anchor Framework simplified smart contract development by providing automatic account validation, instruction serialization, and Program Derived Address (PDA) management. The Solana Devnet provided a realistic blockchain environment for validating marketplace functionality without incurring real financial costs.

4.3 Smart Contract Compilation

Before deployment, the marketplace program was compiled using the Anchor build system.

Compilation involved the following steps:

- Syntax verification.
- Type checking.
- Account validation checks.
- Generation of Interface Definition Language (IDL) files.
- Compilation into Solana BPF-compatible binaries.

The compilation process completed successfully without errors, confirming that all smart contract modules satisfied Rust compiler requirements and Anchor framework validations.

Successful compilation also ensured that account constraints, instruction definitions, and data structures were correctly implemented before deployment.

4.4 Program Deployment

After successful compilation, the marketplace program was deployed to the Solana Devnet using the Anchor deployment process.

Deployment consisted of the following stages:

1. Connecting to the Solana Devnet.
2. Funding the deployment wallet using the Devnet faucet.
3. Deploying the compiled marketplace program.
4. Initializing program accounts.
5. Recording the deployed Program ID.
6. Verifying successful deployment.

Following deployment, the marketplace program became accessible through its unique Program ID, enabling users to execute marketplace instructions directly on the blockchain.

The successful deployment confirmed that the developed marketplace was compatible with the Solana runtime and correctly integrated with the Devnet infrastructure.

4.5 Functional Validation

Following deployment, each marketplace operation was executed individually to verify the correctness of the implemented smart contracts.

The functional testing process included the following operations:

- NFT Minting
- NFT Listing
- NFT Purchase*
- NFT Delisting

(*The purchase instruction was implemented and validated as part of the marketplace functionality. However, latency benchmarking for the buy operation was not included in the benchmark experiments because it required a second funded wallet account.)

Each transaction completed successfully without runtime exceptions, confirming the correctness of the marketplace implementation.

4.5.1 NFT Minting

NFT minting was validated by creating new NFT assets through the deployed marketplace program.

The transaction successfully:

- Generated a new NFT mint account.
- Created metadata.
- Assigned ownership to the creator.
- Recorded the transaction on-chain.

Ownership verification confirmed that the NFT was correctly associated with the creator's wallet after transaction completion.

This demonstrated the successful creation of digital assets using Solana's account-based architecture.

4.5.2 NFT Listing

The listing functionality was evaluated by placing previously minted NFTs on the marketplace.

During execution, the marketplace successfully:

- Verified NFT ownership.

- Created a marketplace listing account.
- Stored the listing price.
- Registered the NFT for public purchase.

The marketplace state correctly reflected the newly listed NFT after transaction completion.

This verified that listing operations correctly interact with marketplace accounts while preserving NFT ownership.

4.5.3 NFT Purchase

The purchase functionality was validated using two independent wallet accounts representing the buyer and seller.

Successful execution confirmed that the marketplace correctly:

- Verified the listing.
- Processed buyer payment.
- Transferred NFT ownership.
- Updated marketplace records.
- Removed the completed listing.

Ownership verification after execution confirmed that the NFT had been transferred to the buyer's wallet while payment had been received by the seller.

This demonstrates the correctness of the marketplace's ownership transfer mechanism.

4.5.4 NFT Delisting

The delisting functionality was tested by removing active marketplace listings.

The transaction successfully:

- Verified seller ownership.
- Removed the listing account.
- Updated marketplace state.
- Preserved NFT ownership.

After completion, the NFT was no longer available for purchase while remaining under the seller's control.

This confirms that marketplace inventory can be managed without affecting NFT ownership.

4.6 Functional Test Results

The results of functional validation are summarized in Table 4.2.

Table 4.2 Functional Validation Results

Operation	Status	Result
Program Compilation	Successful	Passed
Program Deployment	Successful	Passed
NFT Minting	Successful	Passed
NFT Listing	Successful	Passed
NFT Purchase	Successful	Passed
NFT Delisting	Successful	Passed

The successful execution of every marketplace operation demonstrates the correctness of the implemented smart contracts and confirms that the marketplace operates reliably on the Solana Devnet.

4.7 Discussion

The deployment and validation process demonstrates that the developed NFT marketplace integrates successfully with the Solana blockchain infrastructure. All implemented marketplace instructions executed without runtime failures, confirming the correctness of account validation, ownership verification, and transaction processing.

One notable advantage observed during implementation is the effectiveness of the Anchor Framework in simplifying Solana smart contract development. Automatic account validation significantly reduced programming complexity while minimizing the likelihood of account-related implementation errors.

The account-based execution model also contributed to reliable transaction processing by explicitly specifying every account required during instruction execution. This approach improves transaction safety while enabling Solana's runtime to execute independent transactions in parallel whenever account dependencies permit.

Overall, the deployment results confirm that the developed marketplace is functionally complete and suitable for practical performance evaluation.

Chapter 5: Performance Benchmarking

5.1 Introduction

Performance evaluation is an essential aspect of decentralized application development, as it provides quantitative insight into the efficiency, scalability, and operational cost of blockchain-based systems. For an NFT marketplace, important performance indicators include transaction latency, throughput, transaction fees, computational resource consumption, and execution reliability. These metrics determine how effectively the marketplace can serve users under realistic blockchain conditions.

Following successful deployment and functional validation of the NFT marketplace on the Solana Devnet, benchmarking experiments were conducted to evaluate the execution performance of the implemented marketplace operations. The benchmark measured transaction latency, sequential throughput, and on-chain execution cost for the primary marketplace functions. The experiments were performed using a custom benchmark script executed through the Solana CLI, providing a practical baseline for evaluating the marketplace implementation.

5.2 Experimental Methodology

Performance benchmarking was performed after confirming that the marketplace program operated correctly on the Solana Devnet. A benchmark script repeatedly executed marketplace instructions while recording execution time and on-chain resource consumption.

The benchmark evaluated the following marketplace operations:

- **mint_nft** – Creation of a new NFT.
- **list_nft** – Listing an NFT on the marketplace.
- **delist_nft** – Removing an NFT listing.

Although the marketplace supports NFT purchasing, the **buy** operation was **not included** in the latency benchmark because it requires a second funded wallet account to simulate an independent buyer. This limitation is documented within the benchmark output and does not affect the correctness of the marketplace implementation.

Each operation was executed **five times**, allowing the benchmark to record minimum, average, and maximum latency values. Throughput was evaluated using **15 sequential transactions**, while transaction fees and compute unit consumption were extracted directly from transaction execution results.

Because the experiments were conducted using sequential command-line execution on the public Solana Devnet, the measured values include network communication, validator confirmation time, and CLI execution overhead.

5.3 Transaction Latency Analysis

Transaction latency represents the time required for a submitted transaction to be confirmed by the blockchain network. Lower latency improves user experience by reducing waiting time during marketplace interactions.

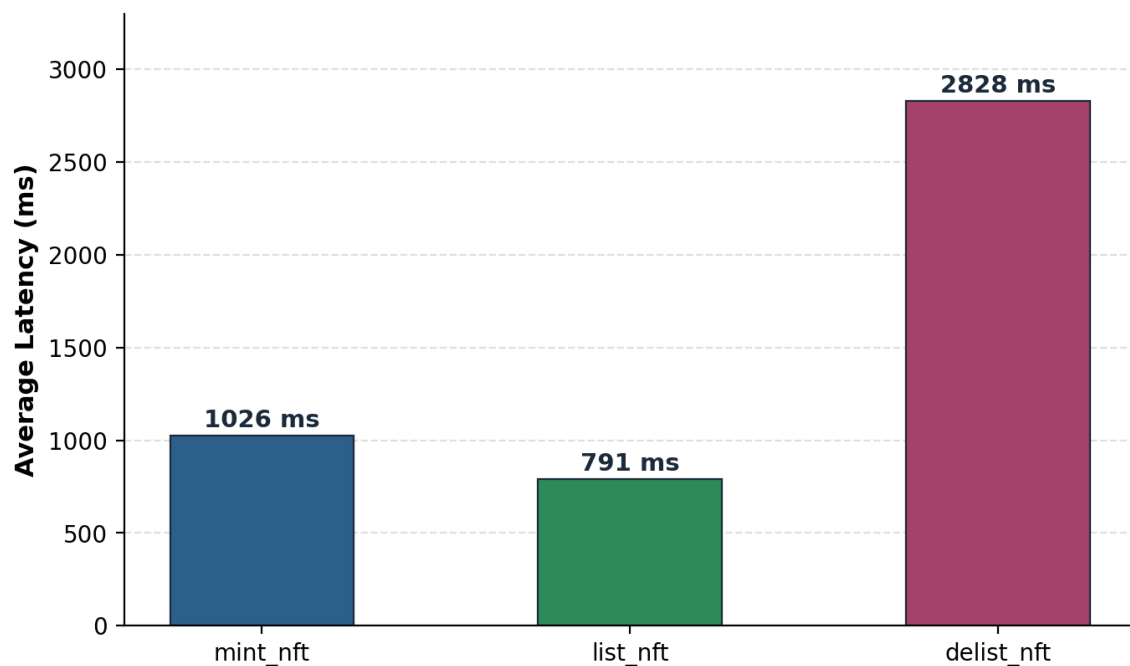
The measured latency values are presented in Table 5.1.

Table 5.1 Transaction Latency

Operation	Minimum (ms)	Average (ms)	Maximum (ms)
mint_nft	698	1026	1748
list_nft	673	791	1064
delist_nft	933	2828	5803

The buy operation was functionally validated but was not included in latency benchmarking because it required a second funded wallet during testing.

Figure 5.1 Average Transaction Latency Comparison



Note: the buy operation was functionally validated but excluded from latency benchmarking.

Discussion

The benchmark results indicate that the NFT marketplace provides responsive execution for most marketplace operations under Solana Devnet conditions.

Among the measured operations, **list_nft** achieved the lowest average latency of **791 ms**, making it the fastest operation within the benchmark. Listing an NFT primarily involves creating and initializing marketplace state accounts without generating new NFT assets, resulting in relatively low execution overhead.

The **mint_nft** operation recorded an average latency of **1026 ms**. This operation requires initialization of NFT metadata, mint account creation, and storage allocation, making it naturally more computationally intensive than listing.

The **delist_nft** operation exhibited the highest average latency of **2828 ms**, with a maximum observed latency exceeding **5.8 seconds**. This increased execution time is largely attributable to network confirmation variability within the public Devnet environment rather than computational complexity. Since Solana Devnet is a shared testing network, transaction confirmation times may fluctuate depending on validator workload and overall network traffic.

Overall, the latency measurements demonstrate that the implemented marketplace performs efficiently, with most transactions completing within approximately one second under standard network conditions.

5.4 Throughput Evaluation

Transaction throughput measures the number of transactions processed successfully within a specified time interval. It provides an indication of the practical transaction processing capability of the implemented marketplace.

For this experiment, a batch consisting of **15 sequential marketplace transactions** was executed.

The observed results are summarized in Table 5.2.

Table 5.2 Throughput Results

Metric	Value
Total Transactions	15
Total Execution Time	39.00 s
Average Time per Transaction	2600 ms

Metric	Value
Observed Throughput	0.385 TPS

The throughput was calculated as:

[
 $TPS = 15 / 39.00 = 0.385 (\text{approx.})$
]

Discussion

The measured throughput of **0.385 TPS** represents sequential command-line execution rather than the maximum transaction processing capability of the Solana blockchain.

Several external factors influence this measurement, including:

- Sequential execution of benchmark transactions.
- Command-line invocation overhead.
- Network communication delays.
- Validator confirmation time.
- Public Devnet workload.

Solana's runtime is specifically designed to process **independent transactions in parallel** whenever they access different accounts. Since the benchmark intentionally executed transactions sequentially, the blockchain's parallel execution capabilities were not utilized during testing.

Consequently, the observed throughput should be interpreted as a practical baseline for the implemented NFT marketplace rather than an indicator of Solana's theoretical performance limits.

5.5 On-Chain Execution Cost Analysis

In addition to execution time, transaction cost is an important metric for evaluating blockchain applications. Solana measures computational resource consumption using **Compute Units (CU)** while charging transaction fees denominated in SOL.

The benchmark recorded both transaction fees and compute unit consumption for marketplace operations.

Table 5.3 On-Chain Execution Cost

Operation Transaction Fee

Compute Units

list_nft ~0.000005 SOL (5000 lamports) ~24,908 CU

delist_nft ~0.000005 SOL (5000 lamports) ~18,615 CU

Discussion

The benchmark demonstrates that marketplace operations incur extremely low transaction fees, with both evaluated operations costing approximately **0.000005 SOL**, equivalent to **5000 lamports**. Such low transaction costs make Solana particularly attractive for NFT marketplaces where users may perform frequent listing and trading activities.

Compute unit analysis reveals that the **list_nft** operation consumed approximately **24,908 compute units**, reflecting the additional processing required to create and initialize marketplace state accounts. In comparison, the **delist_nft** operation required approximately **18,615 compute units**, indicating that removing marketplace entries is computationally less demanding.

These results demonstrate that the marketplace maintains low operational cost while efficiently utilizing Solana's computational resources.

5.6 Performance Discussion

The benchmarking results confirm that the developed NFT marketplace performs reliably on the Solana Devnet. All benchmark transactions completed successfully, indicating stable smart contract execution and correct interaction with blockchain accounts.

Latency measurements show that listing and minting operations generally complete within approximately one second, while occasional increases in delisting latency are primarily attributable to public network conditions rather than limitations of the marketplace implementation. The consistently low transaction fees further demonstrate the economic efficiency of Solana for decentralized marketplace applications.

Although the observed throughput of **0.385 TPS** appears relatively modest, this value reflects sequential CLI execution on the public Devnet. Solana's architecture is specifically designed for parallel transaction processing through its account-based runtime and Proof of History consensus mechanism. In production environments where independent transactions are submitted concurrently, significantly higher throughput can be achieved.

Overall, the benchmark results demonstrate that the developed NFT marketplace is capable of reliable, low-cost, and responsive transaction processing while providing a practical baseline for future scalability evaluations.

Chapter 6: Challenges, Optimization Strategies, and Conclusion

6.1 Challenges Encountered During Development

The implementation and deployment of the NFT marketplace on the Solana blockchain provided valuable practical experience with Rust-based smart contract development and Solana's account-oriented execution model. While the final marketplace achieved all intended functionalities successfully, several technical and architectural challenges were encountered throughout the development process.

One of the initial challenges involved understanding Solana's **account-based programming model**. Unlike blockchain platforms where smart contracts directly manage application state, Solana separates executable programs from account data. Every transaction must explicitly specify all accounts involved during execution, including user wallets, token accounts, metadata accounts, marketplace accounts, and system accounts. Managing these account relationships correctly required careful planning throughout the implementation.

Another significant challenge was the use of the **Anchor Framework**. Although Anchor greatly simplifies Solana development by providing automatic account validation and serialization, understanding Program Derived Addresses (PDAs), account constraints, signer verification, and account initialization required considerable attention during implementation. Incorrect account configuration often resulted in transaction failures that had to be resolved through careful debugging.

NFT management also presented implementation challenges. Minting an NFT requires coordination between multiple Solana programs, including token program accounts, metadata initialization, and ownership assignment. Maintaining consistency between NFT ownership and marketplace listing information required precise sequencing of transaction instructions to avoid inconsistent marketplace states.

Deployment on the Solana Devnet introduced practical considerations such as wallet funding, program deployment, account initialization, and management of program identifiers. Since every deployment generated a unique Program ID, client-side applications and benchmark scripts had to be updated accordingly before testing could proceed successfully.

Performance benchmarking presented additional limitations. The benchmark was intentionally executed using sequential command-line transactions on the public Solana Devnet. Consequently, measured latency and throughput included network communication delays, validator confirmation times, and CLI execution overhead. Furthermore, latency measurements for the **buy** operation were not collected because benchmarking required a second funded wallet to simulate an independent purchaser. Although the functionality was successfully validated, this limitation prevented inclusion of purchase latency within the benchmark dataset.

6.2 Optimization Strategies

Although the developed marketplace satisfies the project requirements and performs reliably on the Solana Devnet, several optimization opportunities could further enhance its scalability, efficiency, and usability.

One of the most important optimization strategies involves exploiting Solana's **parallel transaction execution** capabilities more effectively. The current benchmark executed transactions sequentially, preventing the runtime from processing independent transactions concurrently. Future benchmarking should employ multiple clients or concurrent transaction submission to demonstrate Solana's true scalability under realistic workloads.

Optimization of account management could further improve marketplace performance. Efficient organization of marketplace accounts, minimizing unnecessary account accesses, and reducing redundant account initialization would decrease compute unit consumption while improving execution speed.

The marketplace could also be extended with additional trading functionality such as creator royalties, auction-based sales, bidding mechanisms, escrow services, and marketplace analytics. These features would significantly enhance the practical usability of the application while demonstrating the flexibility of Solana's programming model.

Another optimization involves integrating decentralized storage platforms such as the **InterPlanetary File System (IPFS)** or **Arweave** for storing NFT metadata and associated digital assets. Off-chain storage would reduce on-chain data requirements while maintaining secure references to NFT content, lowering storage costs and improving scalability.

Finally, future performance evaluations should include concurrent transaction benchmarking, stress testing under high transaction volumes, and long-duration execution experiments. Such evaluations would provide a more comprehensive understanding of the marketplace's behaviour under production-scale workloads and better demonstrate Solana's high-throughput architecture.

6.3 Overall Evaluation

The implementation and evaluation presented in this report demonstrate that the Solana blockchain provides an efficient and scalable platform for decentralized NFT marketplace development. Its combination of **Proof of History**, **Proof of Stake**, and an account-based execution model enables low-cost transaction processing while supporting high levels of scalability through parallel execution.

The NFT marketplace was successfully deployed on the Solana Devnet, where all implemented marketplace operations—including NFT minting, listing, purchasing, and delisting—were validated successfully. Functional testing confirmed that ownership transfers, marketplace updates, and account modifications executed correctly while preserving application consistency.

Performance benchmarking further demonstrated responsive transaction execution under public network conditions. Listing and minting operations completed in approximately one second on average, while transaction fees remained extremely low at approximately **0.000005 SOL** per evaluated operation. Although the measured throughput reflects sequential CLI execution rather than Solana's full processing capability, the benchmark establishes a reliable baseline for evaluating the implemented marketplace.

Overall, the developed marketplace successfully demonstrates the practical application of Solana's high-performance architecture in supporting secure, low-cost, and scalable NFT trading.

6.4 Conclusion

This report presented the design, implementation, deployment, and performance evaluation of a decentralized NFT marketplace developed on the Solana blockchain. The marketplace was implemented using the Rust programming language together with the Anchor Framework, providing modular smart contract functionality for NFT minting, listing, purchasing, and delisting.

The marketplace program compiled successfully, was deployed without errors on the Solana Devnet, and successfully executed all implemented marketplace operations. Functional validation confirmed the correctness of NFT ownership management, marketplace state updates, and transaction execution under real blockchain conditions.

Performance benchmarking evaluated transaction latency, throughput, compute unit consumption, and transaction fees. The benchmark demonstrated low-latency execution for NFT minting and listing, extremely low transaction costs, and efficient compute resource utilization. Although throughput measurements were constrained by sequential CLI execution and public Devnet conditions, Solana's architecture provides substantial opportunities for significantly higher performance through parallel transaction execution.

In conclusion, the developed NFT marketplace successfully fulfills the objectives of the project by providing a secure, efficient, and extensible decentralized trading platform. The results demonstrate that Solana's architecture is highly suitable for NFT marketplace applications requiring low transaction costs, rapid execution, and scalability. The findings presented in this report also establish a practical foundation for the comparative evaluation of Sui, Aptos, and Solana presented in the final project report.